

Runtime-Governed Systems

White Paper · Version 1.0 · February 18, 2026

This white paper defines the LAIRS (Local Autonomous Intelligence and Reasoning System) standard for Runtime-Governed Systems — computing environments in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints during operation. The document describes the governance model, required architectural layers, artifact specifications, and compliance criteria.

■ Platform Status Notice

LAIRS is an active research and architectural exploration platform. It is **not production ready** and is intended for controlled experimentation. Components marked *In Development* are under active work and subject to change. Do not deploy in production environments without explicit authorization.

1. Definition

A **Runtime-Governed System** is a computing environment in which authority, behavior, and state changes are continuously validated through cryptographic proof, independent verification, and enforceable policy constraints *during operation* — not only at deployment time.

Unlike traditional systems that rely on static configuration trust, runtime-governed systems maintain persistent assurance of correctness, authorization, and integrity while executing.

Trust is not assumed. Trust is continuously proven.

2. Problems Addressed

Modern distributed and autonomous systems suffer from several structural weaknesses that runtime governance directly addresses by introducing verifiable authority loops:

- Configuration and policy drift over time
- Loss of authority provenance for system actions
- Inability to independently verify state legitimacy
- Weak coupling between decision and enforcement layers
- Over-reliance on perimeter or identity-only security models
- Lack of provable governance for automated or AI-driven changes

3. LAIRS Runtime Governance Model

The LAIRS architecture defines three primary governance layers, each with distinct responsibilities and a specific trust question answered at runtime.

LAYER 1 Governance Authority e.g. Metatron's Gate › Issue signed governance artifacts (GATs) › Maintain immutable decision ledger › Define quorum and approval requirements › Anchor trust roots › Provide audit provenance "Who authorized this action?"	LAYER 2 Trust Fabric e.g. SentiNet › Independent artifact verification › Node binding enforcement › Signature and fingerprint validation › Last-Known-Good state protection › Cross-node trust propagation "Can this authority be trusted here?"	LAYER 3 Enforcement e.g. Sentinel Agents › Runtime policy enforcement › Integrity monitoring › Behavior control › Evidence generation › Local protective response "Is the system doing what was authorized?"
---	--	--

4. Governance Artifact Requirement

A runtime-governed system must rely on explicit **Governance Artifacts** to authorize all actions. In LAIRS this artifact is formally designated the **GAT — Governance Artifact Token**. Without a valid artifact, enforcement actions must fail closed.

Minimum required properties of a GAT:

Property	Purpose
----------	---------

Cryptographically signed	Ensures tamper-evident authenticity of the issuing authority
Time bounded	Tokens expire; stale authority cannot be replayed
Context scoped	Authority is limited to specific action types and contexts
Node bound	Tokens bind to specific infrastructure nodes or identities
Independently verifiable	Third-party validation without querying the issuer
Ledger anchored	Linked to an immutable decision log for provenance
Authority metadata	Records who issued, under what policy, with what approval
Action intent hash	Binds the token to the exact action being authorized

5. Runtime Verification Requirements

A system qualifies as runtime-governed only if all of the following conditions hold continuously during operation:

- Authority artifacts are validated during execution, not only at deployment
- Verification occurs independently from the issuing authority
- Enforcement modules refuse actions when validation fails
- Expired or revoked authority cannot execute
- State rollback or Last-Known-Good protection exists
- Evidence of decisions is recorded immutably
- Nodes can prove their operational state to external observers

6. Governed Adversarial Evolution (GAE)

LAIRS extends runtime governance through **Governed Adversarial Evolution (GAE)**, enabling continuous system improvement without sacrificing governance integrity:

Component	Description
Red Team Pressure	Adversarial testing inside defined policy envelopes
Blue Team Defense	Adaptive defense responses within governed boundaries
Purple Team Synthesis	Coordinated insight combining offensive and defensive findings

AI Advisory	AI recommendations without autonomous authority to act
Human Approval Gates	Cryptographic human sign-off required for all changes
Replayable Scenarios	Scenarios recorded for regression and auditability

7. Observe-Only Safety Mode

Runtime-governed systems support staged deployment through **Observe-Only Mode**, enabling trust calibration before full enforcement is activated:

- Governance artifacts are issued but non-enforcing
- Verification pipeline is fully active
- Evidence is generated for all decisions
- No destructive or state-modifying actions permitted
- Provides a safe validation environment before live enforcement

8. Properties of a Runtime-Governed System

A compliant system demonstrates all of the following:

- Verifiable authority provenance
- Continuous policy validation
- Cryptographic trust anchors
- Independent enforcement capability
- Deterministic auditability
- Fail-closed behavior under uncertainty
- Bounded autonomy
- Human override capability
- Recoverable safe state
- Cross-node trust coherence

9. Comparison With Traditional Models

Traditional Security	Runtime Governance (LAIRS)
Static configuration trust	Continuous verification
Identity-based authorization	Artifact-based authority
Centralized control	Distributed verification

Logs after the fact	Proof during execution
Implicit trust boundaries	Explicit trust contracts
Reactive response	Governed adaptation

10. LAIRS Standard Compliance Summary

A system meets the LAIRS Runtime Governance Standard if it:

- Uses cryptographically signed governance artifacts (GATs)
 - Performs independent runtime verification across all enforcement nodes
 - Enforces policy through verifiable agents with fail-closed behavior
 - Maintains immutable decision provenance via anchored ledger
 - Supports safe recovery and Last-Known-Good state rollback
 - Enables governed evolution without authority loss through GAE
 - Implements multi-authority quorum governance for high-assurance decisions
-

11. Benefits

- Reduced configuration drift risk through continuous artifact-based validation
 - Provable compliance with immutable, independently verifiable decision records
 - Safer automation and AI integration via bounded autonomy and human approval gates
 - Strong forensic capability with evidence generated at every enforcement decision
 - Independent trust validation without reliance on centralized authority at runtime
 - Resilient distributed operation with cross-node trust coherence
 - Controlled autonomy that evolves without sacrificing governance integrity
-

12. Future Directions

The following capabilities are on the LAIRS roadmap for future development:

- Hardware root attestation integration for silicon-level trust anchoring
- Cross-organization trust federation enabling governed interoperability
- Autonomous infrastructure governance for large-scale self-managing systems
- Regulated AI control environments with LAIRS-compliant oversight layers

This catalog reflects the current public architecture as of February 18, 2026. Components are classified as **Implemented** (operational in the current two-node deployment) or **In Development** (under active design or integration). The Human Master Administrator retains ultimate authority over all components at all times.

✓ **Implemented** — Operational in current deployment

● **In Development** — Under active design or integration

Implemented Components

LAIRS Core

Role: Cognitive runtime and orchestration layer

✓ **Implemented** — Operational

- › Runtime coordination and event routing between all modules
- › Identity, memory, and state management
- › Policy hooks, audit logging, and explainability
- › Local-first execution — no external dependency for core operation
- › No autonomous irreversible actions without human approval

SentiNet Fabric

Role: Zero-trust secure networking and governance fabric

✓ **Implemented** — Operational

- › Zone-based segmentation with default-deny communication model
- › Internal DNS and PKI anchoring
- › Controlled bridge modes between trust zones
- › Every module treated as untrusted until verified
- › Policy enforcement occurs at the fabric layer
- › Last Known Good (LKG) state validation

Metatron's Gate

Role: Quorum governance and system authority control

✓ Implemented — Operational

- › Explicit intent declaration required for all sensitive actions
- › Multi-party quorum enforcement: Machine / AI / Human
- › Cryptographic approval and denial tracking
- › Observe-only and enforcement modes
- › Prevents unilateral system modification by design
- › Append-only audit trail with causal chaining

Sentinel Agent

Role: Host-level integrity, telemetry, and local attestation

✓ Implemented — Operational

- › Host-level monitoring and integrity checks
- › Secure telemetry to LAIRS Core
- › Foundation module for trust establishment
- › Currently operating in observe mode

AiCraft

Role: Local AI training and lifecycle governance

✓ Implemented — Functional for controlled training

- › Dataset ingestion and validation
- › Model training and evaluation
- › Versioned model artifacts
- › Transparency into training behavior
- › Explicit datasets only — no hidden learning
- › Human-directed experimentation

In Development

Sentinel Gate

Role: Policy enforcement and controlled access (Gated Community)

● In Development

- › Entry control point into SentiNet fabric
- › Policy validation at ingress
- › Trust scoring and rejection handling
- › Fail-closed by design

Sentinel AV

Role: Malware detection and quarantine

● In Development

- › Local scanning without cloud dependency
- › Quarantine workflows
- › Integration with Sentinel Agent and Sentinel Gate

Sentinel Forge

Role: Human-approved governed system evolution

● In Development

- › Detects issues and performance degradation
- › Generates candidate patches for human review
- › Stages changes with full rollback capability
- › Requires cryptographic human approval before application
- › Changes applied only at reboot boundaries
- › Advisory only — cannot self-modify autonomously

Leave No Trace (LNT)

Role: Data minimization and forensic hygiene

● In Development

- › Log scrubbing and data retention control
- › Ephemeral runtime handling
- › Privacy-preserving operations
- › Minimizes data footprint without compromising audit integrity

Guiding Philosophy: Digital Sovereignty through Local Intelligence, Governed Evolution, and Zero Trust by Default. LAIRS is designed to empower operators, not replace them. Intelligence is local. Authority remains human. Evolution is deliberate.